C# & .NET Framework Master Examination

Course Syllabus: .NET Environment, Visual Studio, Variables, Conditions, Loops, Lists/Arrays, File I/O, OOP, Stack vs Heap

Total Assessment Weight: 100 Questions (30 MCQ | 20 Essay | 30 Code Writing | 20 Code Correction)

Instructions for Candidates:
1. This exam consists of **100 total questions** divided into 4 main sections.
2. Answer all questions thoroughly based on standard C# language specifications and .NET Framework mechanics.
3. For coding questions, ensure proper syntax, type declarations, and control flow.
4. Full Answer Key & Solutions are provided at the end of the paper for self-grading.

Section 1: Multiple Choice Questions (30 Items — 1 Mark Each)

1. What component of the .NET Framework is responsible for managing code execution, memory, and garbage collection?

A) Just-In-Time (JIT) Compiler
B) Common Language Runtime (CLR)
C) Base Class Library (BCL)
D) Visual Studio IDE

2. Which executable tool converts Intermediate Language (IL) code into native machine instructions at runtime?

A) CLR Garbage Collector
B) Roslyn Compiler
C) Just-In-Time (JIT) Compiler
D) MSBuild

3. In Visual Studio Community, what is the top-level container that groups one or more related projects together?

A) Namespace
B) Assembly
C) Solution
D) Working Directory

4. Which C# data type is a 64-bit signed integer?

A) int
B) short
C) long
D) double

5. What is the correct syntax to explicitly cast a double variable `val = 9.8` to an integer?

A) int x = Convert.ToInt(val);
B) int x = (int)val;
C) int x = val.ToInt();
D) int x = parse(val);

6. What is the result of the integer division expression `int result = 7 / 2;` in C#?

A) 3.5
B) 3
C) 4
D) 3.0

7. Which method is used to convert user keyboard input from `Console.ReadLine()` into an integer?

A) int.Parse()
B) Console.ReadInt()
C) Convert.ToInteger()
D) int.Convert()

8. Which operator represents logical AND in C# conditional expressions?

A) &
B) AND
C) &&
D) ||

9. In a `switch` statement, which keyword handles execution when no matching `case` is met?

A) else
B) default
C) break
D) fallback

10. What exception occurs when accessing an array element at index 5 when the array length is 5?

A) NullReferenceException
B) IndexOutOfRangeException
C) ArgumentOutOfRangeException
D) OverflowException

11. What loop structure guarantees that its body executes at least once before testing the condition?

A) for loop
B) while loop
C) do-while loop
D) foreach loop

12. Which loop statement immediately terminates the loop and jumps execution to the statement following the loop?

A) continue
B) return
C) break
D) exit

13. Which statement skips the remainder of the current loop iteration and moves to the next iteration?

A) break
B) continue
C) skip
D) pass

14. How do you declare a fixed-size array of 10 floating-point numbers in C#?

A) float[] arr = new float[10];
B) Array arr = new Array(10);
C) float arr[10];
D) List<float> arr = new List(10);

15. What property returns the total number of elements contained in a C# `List`?

A) Length
B) Size
C) Count
D) Capacity

16. Which method appends a new item to the end of a `List`?

A) .Push()
B) .Add()
C) .Append()
D) .Insert()

17. Where are C# value types (such as `int`, `bool`, `struct`) typically stored in memory when declared as local variables?

A) Managed Heap
B) Stack
C) Global Registry
D) Hard Disk Cache

18. Where are C# reference types (such as `class` instances and arrays) allocated in memory?

A) Stack
B) CPU Registers
C) Managed Heap
D) Read-Only Segment

19. What keyword is used to create a new instance of a class in C#?

A) alloc
B) create
C) new
D) construct

20. What is the default access modifier for members inside a C# class if no modifier is explicitly specified?

A) public
B) private
C) protected
D) internal

21. Which object-oriented principle refers to restricting direct access to object fields and exposing controlled methods/properties?

A) Polymorphism
B) Inheritance
C) Encapsulation

D) Abstraction

22. What special method is automatically invoked when an object is instantiated to initialize its state?

A) Destructor
B) Initializer
C) Constructor
D) Static Block

23. Which class in the `System.IO` namespace is used to read text characters from a file stream line-by-line?

A) FileOutput
B) StreamReader
C) TextWriter
D) BinaryReader

24. Which class in the `[suspicious link removed]` namespace is used to write text data to a file stream?

A) StreamWriter
B) FileReader
C) ConsoleWriter
D) DataAppender

25. What block in a C# error handling structure contains code that always executes, regardless of whether an exception occurred?

A) try
B) catch
C) finally
D) error

26. What keyword is used to throw an explicit exception programmatically in C#?

A) raise
B) throw
C) error
D) dispatch

27. What is the output of `Console.WriteLine(10 % 3);`?

A) 3
B) 3.33
C) 1
D) 0

28. In a class, what automatic feature combines a private backing field with getter and setter methods?

A) Auto-implemented Property (`{ get; set; }`)
B) Constructor
C) Delegate
D) Struct

29. Which C# collection type allows dynamic resizing and belongs to `System.Collections.Generic`?

A) System.Array
B) List<T>
C) FixedVector

D) Buffer

30. What happens when two reference variables point to the same object on the Heap and one variable modifies a property?

A) Only that variable sees the change
B) The change is reflected across both variables
C) A runtime crash occurs
D) The second variable is set to null

Section 2: Conceptual & Short Essay Questions (20 Items)

31. Explain the step-by-step compilation and execution lifecycle of a C# application, from source code to machine code execution via CLR and JIT.

[Write your answer below — 3-5 sentences]

32. Compare the roles of a Solution (`.sln`) and a Project (`.csproj`) in Microsoft Visual Studio Community.

[Write your answer below — 3-5 sentences]

33. Define 'Managed Code' in the context of the .NET Framework and contrast it with unmanaged C/C++ code.

[Write your answer below — 3-5 sentences]

34. Explain the difference between implicit type conversion (widening) and explicit type casting (narrowing) in C#, giving examples of when explicit casting can cause data loss.

[Write your answer below — 3-5 sentences]

35. Differentiate between integer division (`5 / 2`) and floating-point division (`5.0 / 2.0` or `(double)5 / 2`) in C#.

[Write your answer below — 3-5 sentences]

36. Explain short-circuit evaluation in C# logical operators (`&&` vs `&` and `||` vs `|`). Why is short-circuiting advantageous?

[Write your answer below — 3-5 sentences]

37. Describe the syntax and execution mechanics of a C# `switch` statement, including the role of `case`, `break`, and `default` labels.

[Write your answer below — 3-5 sentences]

38. Compare a `for` loop, a `while` loop, and a `do-while` loop. When should a developer choose each loop structure?

[Write your answer below — 3-5 sentences]

39. Explain the exact operational difference between the `break` statement and the `continue` statement within loop constructs.

[Write your answer below — 3-5 sentences]

40. Contrast a fixed-size Array (`int[]`) with a dynamic Generic List (`List`) in terms of memory allocation, resizing capability, and syntax.

[Write your answer below — 3-5 sentences]

41. Explain what causes an `IndexOutOfRangeException` in C# arrays and how defensive programming can prevent it.

[Write your answer below — 3-5 sentences]

42. Detailed Memory Model Analysis: Explain the Stack vs. Heap allocation rules for Value Types (`int`, `double`, `bool`, `struct`) versus Reference Types (`class`, `string`, arrays).

[Write your answer below — 3-5 sentences]

43. Describe what happens in memory during assignment for value types (`int b = a;`) versus reference types (`Person p2 = p1;`).

[Write your answer below — 3-5 sentences]

44. What is Garbage Collection (GC) in .NET? Explain how GC automatically manages Heap memory and cleans up unreferenced objects.

[Write your answer below — 3-5 sentences]

45. Define the principle of Encapsulation in Object-Oriented Programming and explain why fields should be declared `private` while providing `public` properties.

[Write your answer below — 3-5 sentences]

46. Explain the purpose of Class Constructors in C#. What is a default constructor versus a parameterized constructor?

[Write your answer below — 3-5 sentences]

47. Differentiate between Method Parameters and Class Instance Fields/Properties. Explain the purpose of the `this` keyword in C# constructors.

[Write your answer below — 3-5 sentences]

48. Explain how text files are read line-by-line using `StreamReader` in C#, including why placing streams inside `using` blocks is essential for resource management.

[Write your answer below — 3-5 sentences]

49. Describe the structure and flow of C# Exception Handling using `try`, `catch`, and `finally` blocks.

[Write your answer below — 3-5 sentences]

50. Explain the difference between a Class (blueprint) and an Object (instance) in C# object-oriented design.

[Write your answer below — 3-5 sentences]

Section 3: Code Writing & Implementation (30 Items)

51. Write a C# snippet that prompts the user for their name and age via `Console.ReadLine()`, parses the age to an `int`, and prints: 'Hello [Name], next year you will be [Age+1]'.

```
// Write your C# solution code here:
```

52. Write a program that reads a double number from console input, converts it to an integer using explicit casting `(int)`, and prints both original and cast values.

```
// Write your C# solution code here:
```

53. Write an `if-else` statement that checks an integer score variable and prints 'Grade A' (>=90), 'Grade B' (70-89), or 'Fail' (<70).

```
// Write your C# solution code here:
```

54. Convert the following `if-else` chain into an equivalent C# `switch` statement: `if (code == 1) Console.WriteLine("Start"); else if (code == 2) Console.WriteLine("Stop"); else Console.WriteLine("Unknown");`

```
// Write your C# solution code here:
```

55. Write a `for` loop that prints all even numbers from 2 to 20 (inclusive) separated by spaces.

```
// Write your C# solution code here:
```

56. Write a `while` loop that repeatedly prompts the user to enter a password until they type 'secret123'.

```
// Write your C# solution code here:
```

57. Write a `do-while` loop that sums positive numbers entered by the user and terminates when the user enters 0.

```
// Write your C# solution code here:
```

58. Write a program that uses a loop and `continue` statement to print all numbers from 1 to 15, skipping multiples of 3.

```
// Write your C# solution code here:
```

59. Write code to declare an array of 5 integers, assign values {10, 20, 30, 40, 50}, and print the sum using a `foreach` loop.

```
// Write your C# solution code here:
```

60. Write a program that finds and prints the maximum value in an array of integers `int[] numbers = { 14, 58, 2, 99, 31 };`.

```
// Write your C# solution code here:
```

61. Write C# code to instantiate a `List`, add 3 country names, remove 1 country name, and print the total list count.

```
// Write your C# solution code here:
```

62. Write a C# program that searches a `List` for a target number provided by user input and prints whether it was found or not.

```
// Write your C# solution code here:
```

63. Write a program that populates a `List` with 4 grade values, calculates the average grade using a loop, and displays the average.

```
// Write your C# solution code here:
```

64. Create a C# `Book` class with public properties `Title` (string) and `Price` (double), and a method `DisplayDetails()`.

```
// Write your C# solution code here:
```

65. Add a custom parameterized constructor to the `Book` class that initializes `Title` and `Price` upon creation.

```
// Write your C# solution code here:
```

66. Create a `BankAccount` class with a private field `balance`. Add a `Deposit(double amount)` method that increases balance only if `amount > 0`.

```
// Write your C# solution code here:
```

67. Add a `GetBalance()` method or public getter property to `BankAccount` that returns the current balance.

```
// Write your C# solution code here:
```

68. Instantiate two `BankAccount` objects in `Main()`, deposit $100 into account 1, deposit $250 into account 2, and print both balances.

```
// Write your C# solution code here:
```

69. Write C# code that writes three lines of text ("Line 1", "Line 2", "Line 3") to a file named 'output.txt' using `StreamWriter`.

```
// Write your C# solution code here:
```

70. Write C# code that reads all text lines from 'output.txt' using `StreamReader` inside a `while` loop and prints them to the console.

```
// Write your C# solution code here:
```

71. Wrap a string-to-int conversion (`int.Parse(userInput)`) inside a `try-catch` block that catches `FormatException` and displays a friendly error message.

```
// Write your C# solution code here:
```

72. Write a method `double CalculateArea(double width, double height)` that returns the area of a rectangle.

```
// Write your C# solution code here:
```

73. Write a method `bool IsEven(int number)` that returns `true` if a number is even, otherwise `false`.

```
// Write your C# solution code here:
```

74. Write code to copy all elements from an array `int[] source = {1, 2, 3};` into a new `List`.

```
// Write your C# solution code here:
```

75. Create a `Student` class with properties `Name` and `Score`. Write a program that filters a `List` and prints names of students scoring >= 50.

```
// Write your C# solution code here:
```

76. Write a program that demonstrates value type behavior by assigning `int a = 10; int b = a; b = 20;` and printing both `a` and `b`.

```
// Write your C# solution code here:
```

77. Write a program that demonstrates reference type behavior by assigning `int[] arr1 = {1, 2}; int[] arr2 = arr1; arr2[0] = 99;` and printing `arr1[0]`.

```
// Write your C# solution code here:
```

78. Write a code snippet using `System.IO.File.Exists("data.txt")` to check if a file exists before attempting to read it.

```
// Write your C# solution code here:
```

79. Write a C# method `void PrintReverse(int[] arr)` that prints the elements of an array in reverse order.

```
// Write your C# solution code here:
```

80. Write a C# program that reads numbers from a text file `numbers.txt` line by line, converts each line to an integer, and calculates their total sum.

```
// Write your C# solution code here:
```

Section 4: Code Correction & Debugging (20 Items)

Identify the syntax, logical, or runtime error in each C# code snippet below and write the corrected code.

Question 81: 81. Bug: Compiler error due to incompatible type assignment.

```
int score = 95.5;
```

Correction / Fixed Code:

Question 82: 82. Bug: Runtime IndexOutOfRangeException on loop termination.

```
int[] numbers = new int[5];
for (int i = 0; i <= numbers.Length; i++) {
numbers[i] = i * 2;
}
```

Correction / Fixed Code:

Question 83: 83. Bug: Compiler error due to unassigned local variable.

```
int total;
Console.WriteLine("Total is: " + total);
```

Correction / Fixed Code:

Question 84: 84. Bug: Incomplete switch case statement causing syntax error.

```
int val = 2;
switch(val) {
case 1: Console.WriteLine("One");
case 2: Console.WriteLine("Two");
}
```

Correction / Fixed Code:

Question 85: 85. Bug: Compiler error attempting to modify foreach iteration variable.

```
List scores = new List { 10, 20, 30 };
foreach (int s in scores) {
s = s + 5;
}
```

Correction / Fixed Code:

Question 86: 86. Bug: NullReferenceException when calling method on uninstantiated object.

```
List names;
names.Add("Alice");
```

Correction / Fixed Code:

Question 87: 87. Bug: Logical error in integer division resulting in 0.0.

```
double average = 5 / 2;
Console.WriteLine(average);
```

Correction / Fixed Code:

Question 88: 88. Bug: Syntax error creating class instance.

```
Car myCar = Car();
```

Correction / Fixed Code:

Question 89: 89. Bug: Access level compiler error attempting to access private field from Main().

```
class Person { string name = "John"; }
// In Main():
Person p = new Person();
Console.WriteLine(p.name);
```

Correction / Fixed Code:

Question 90: 90. Bug: Missing return statement in method path.

```
public int Multiply(int a, int b) {
int result = a * b;
}
```

Correction / Fixed Code:

Question 91: 91. Bug: Unhandled exception when file does not exist.

```
string content = System.IO.File.ReadAllText("missing.txt");
```

Correction / Fixed Code:

Question 92: 92. Bug: Incorrect syntax for declaring a generic List.

```
List numbers = new List();
```

Correction / Fixed Code:

Question 93: 93. Bug: Infinite loop due to missing loop counter update.

```
int count = 0;
while (count < 5) {
Console.WriteLine("Running...");
}
```

Correction / Fixed Code:

Question 94: 94. Bug: Misplaced semicolon terminating if conditional prematurely.

```
int x = 10;
if (x > 5);
{
Console.WriteLine("Greater");
}
```

Correction / Fixed Code:

Question 95: 95. Bug: Constructor name does not match class name.

```
class Student {
public void StudentConstructor(string n) { }
}
```

Correction / Fixed Code:

Question 96: 96. Bug: Invalid string comparison using single assignment operator.

```
string status = "active";
if (status = "active") {
Console.WriteLine("OK");
}
```

Correction / Fixed Code:

Question 97: 97. Bug: Missing catch block in exception handling structure.

```
try {
int num = int.Parse("abc");
} catch {
// Empty block without exception parameter
}
```

Correction / Fixed Code:

Question 98: 98. Bug: Invalid array index access using negative integer.

```
int[] arr = { 1, 2, 3 };
Console.WriteLine(arr[-1]);
```

Correction / Fixed Code:

Question 99: 99. Bug: Resource leak due to unclosed StreamWriter file lock.

```
StreamWriter writer = new StreamWriter("log.txt");
writer.WriteLine("Event log");
// File remains locked
```

Correction / Fixed Code:

Question 100: 100. Bug: Reference type copy confusion.

```
class Point { public int X; }
Point p1 = new Point { X = 5 };
Point p2 = p1;
p2.X = 10;
// Developer expected p1.X to remain 5!
```

Correction / Fixed Code:

Comprehensive Answer Key & Solution Guide

Official Solutions for Questions 1 through 100

Section 1 Answers (MCQ 1-30)

1. B (CLR) | 2. C (JIT) | 3. C (Solution) | 4. C (long) | 5. B ((int)val) | 6. B (3)

7. A (int.Parse) | 8. C (&&) | 9. B (default) | 10. B (IndexOutOfRangeException)

11. C (do-while) | 12. C (break) | 13. B (continue) | 14. A (new float[10]) | 15. C (Count)

16. B (.Add()) | 17. B (Stack) | 18. C (Managed Heap) | 19. C (new) | 20. B (private)

21. C (Encapsulation) | 22. C (Constructor) | 23. B (StreamReader) | 24. A (StreamWriter)

25. C (finally) | 26. B (throw) | 27. C (1) | 28. A (Auto Property) | 29. B (List) | 30. B (Reflected across both)

Section 2 Answers Highlights (Conceptual 31-50)

31. Source code (.cs) -> Roslyn Compiler -> CIL/IL (.dll/.exe) -> CLR loads assembly -> JIT compiles IL to native machine code on execution.

32. Solution (.sln) is a workspace container holding multiple related Projects (.csproj), where each project compiles into an assembly.

33. Managed code runs under CLR execution control (garbage collection, type safety, boundary checks). Unmanaged code runs directly on OS hardware.

34. Implicit casting is safe automatic conversion (e.g. int to double). Explicit casting (double to int) requires `(int)` and truncates decimals.

35. `5 / 2` performs integer division returning `2`. `5.0 / 2.0` or `(double)5 / 2` performs floating-point division returning `2.5`.

36. `&&` and `||` evaluate the right side only if necessary. Short-circuiting prevents unnecessary CPU operations and avoids null exceptions.

37. `switch` tests a variable against `case` values. `break` stops fall-through. `default` executes if no case matches.

38. `for`: known iteration count. `while`: condition-driven (0+ times). `do-while`: condition-driven guaranteed at least 1 execution.

39. `break` terminates loop entirely. `continue` skips the remaining body lines of the current iteration and jumps to the next condition evaluation.

40. Arrays (`int[]`) have fixed size on allocation. Lists (`List`) dynamically expand as elements are added using an underlying buffer.

41. Occurs when index `< 0` or `>= array.Length`. Prevent by checking `if (index >= 0 && index < arr.Length)` before indexing.

42. Value types store raw values on Stack (fast, short-lived). Reference types store actual object instances on Heap; Stack holds reference address pointer.

43. Value type copy creates independent value copy. Reference type copy copies the memory address pointer, so both variables reference the same Heap object.

44. .NET GC periodically identifies Heap objects with no active references and reclaims their memory automatically.

45. Encapsulation hides private fields and exposes public properties (`get; set;`) to enforce validation and prevent illegal direct state modification.

46. Constructors initialize instance state. Default constructor takes 0 parameters; parameterized constructor sets initial property values.

47. `this` refers to the current class instance, disambiguating instance fields from constructor parameter names (e.g., `this.name = name;`).

48. `StreamReader.ReadLine()` reads text line by line. `using` blocks ensure streams are properly closed and disposed automatically upon exit.

49. `try` encloses risky code, `catch` handles specific runtime exceptions, and `finally` runs cleanup code regardless of success or error.

50. A Class is a blueprint/template defining fields and methods. An Object is a concrete instance allocated in Heap memory with state.

Section 3 Answers Highlights (Coding 51-80)

51. `string name = Console.ReadLine(); int age = int.Parse(Console.ReadLine()); Console.WriteLine($"Hello {name}, next year you will be {age+1}");`

54. `switch(code) { case 1: Console.WriteLine("Start"); break; case 2: Console.WriteLine("Stop"); break; default: Console.WriteLine("Unknown"); break; }`

59. `int[] arr = {10,20,30,40,50}; int sum = 0; foreach(int n in arr) sum += n; Console.WriteLine(sum);`

64-65. `public class Book { public string Title {get; set;} public double Price {get; set;} public Book(string t, double p) { Title=t; Price=p; } }`

66-67. `public class BankAccount { private double balance; public void Deposit(double a) { if(a>0) balance+=a; } public double GetBalance() => balance; }`

69. `using(StreamWriter sw = new StreamWriter("output.txt")) { sw.WriteLine("Line 1"); sw.WriteLine("Line 2"); }`

71. `try { int num = int.Parse(userInput); } catch(FormatException) { Console.WriteLine("Invalid integer format"); }`

75. `foreach(var s in students) { if(s.Score >= 50) Console.WriteLine(s.Name); }`

Section 4 Answers Highlights (Debugging 81-100)

81 Fix: `double score = 95.5;` or `int score = (int)95.5;`

82 Fix: Change condition from `i <= numbers.Length` to `i < numbers.Length`.

83 Fix: Initialize variable before use: `int total = 0;`.

84 Fix: Add missing `break;` statements to each switch case.

85 Fix: Cannot modify foreach loop variable `s`. Use a standard `for` loop or modify underlying collection.

86 Fix: Instantiate list before adding elements: `List names = new List();`.

87 Fix: Use floating-point literal: `double average = 5.0 / 2;` (results in 2.5).

88 Fix: Add missing `new` keyword: `Car myCar = new Car();`.

89 Fix: Make field public or use property: `public string name = "John";`.

90 Fix: Add return statement: `return result;` at end of method body.

93 Fix: Increment counter inside loop body: `count++;`.

94 Fix: Remove errant semicolon directly following `if(x > 5)`.

96 Fix: Use comparison operator `==` instead of assignment `=`: `if (status == "active")`.

98 Fix: C# array indexing starts at 0 up to `Length-1`. Negative indexing is invalid.

99 Fix: Wrap `StreamWriter` in a `using` block or call `writer.Close();`.

100 Explanation: `Point` is a reference class; `p2` points to the same object as `p1`. Modifying `p2.X` modifies `p1.X`. Use `struct Point` or create a new instance for value behavior.